

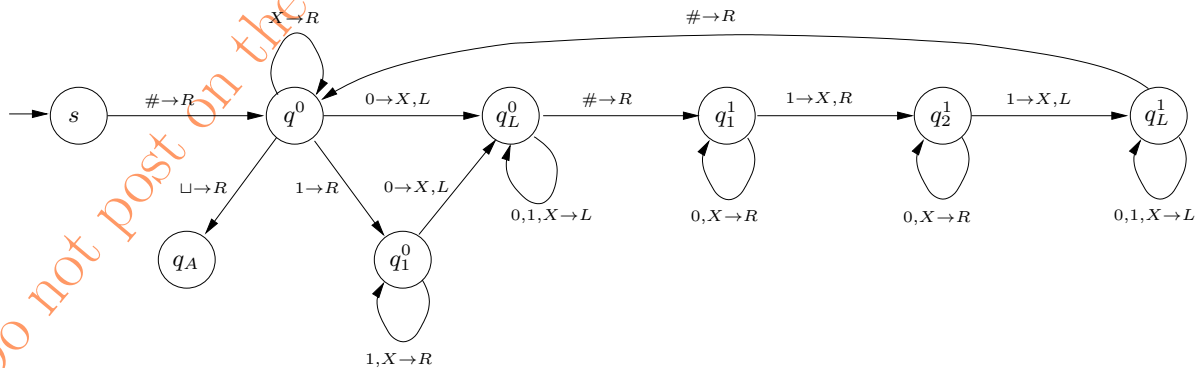
Solutions for Homework Assignment #2

Answer to Question 1. At a high level, the idea is for the TM to sweep to the right until it finds a 0, which it crosses out. It then moves back to the leftmost cell and sweeps to the right again, now looking for two 1s to cross out. Once it has done that, it repeats this process until there are no 0s or 1s left on the tape (i.e., all of them have been crossed out), at which point it accepts. The TM rejects if it is unable to find a 0 but there are still 1s left on the tape, or it is unable to find two 1s after having crossed a 0.

We now describe the TM in more detail. Its tape alphabet consists of the input alphabet $\{\#, 0, 1\}$, the blank symbol $\sqcup$, and a new symbol X that we use to “cross out” 0s and 1s. The TM operates as follows:

1. If the symbol it is scanning is $\#$, it moves to the right; otherwise it rejects right off the bat.
2. It is now about to start looking for the leftmost 0. If it sees a blank, it means that twice as many 1s as 0s have been crossed out, and there are no more 0s or 1s left on the tape, so it accepts. If not, it moves to the right looking for a 0, skipping over X s and 1s, but “remembering” if it has seen a 1 (we will see why in the next item).
3. As soon as it finds a 0, it replaces it with an X , and keeps moving to the left looking for the leftmost $\#$, skipping over all other symbols (0, 1, X). If the TM does not find a 0, but it has seen a 1, the input string has more than twice as many 1s as 0s (because so far the TM has crossed out twice as many 1s as 0s, and there still 1s but no 0s remaining on the tape), so it rejects.
4. Once the TM has moved back to the leftmost cell, i.e., the one containing $\#$, it moves right looking for a 1, skipping over any 0s and X s that it finds on the way. If it finds a 1, it replaces it with an X and continues the rightward sweep, looking for a second 1. If it finds the second 1, it replaces it with an X as well. If it fails to find the first or the second 1, it rejects (because there are fewer than twice as many 1s as there are 0s).
5. Having found and crossed out two 1s for the last 0 it crossed out, the TM moves back to the leftmost cell, skipping over all symbols other than the $\#$ it is looking for.
6. Once it finds the leftmost cell containing the $\#$, the TM moves right, and resumes its operation from Step 2.

Formally, the TM described informally above is $M = (Q, \Sigma, \Gamma, \delta, s, q_A, q_R)$, where Σ is as given in the question, $\Gamma = \Sigma \cup \{\sqcup, X\}$, the initial state is s , the accept and reject states are q_A and q_R , respectively, and the states and transition function are shown diagrammatically below. Any missing transitions implicitly lead to the reject state, which is not shown for clarity.



To relate the formal description to the informal one given above:

- s is the initial state of the TM;
- q^0 is the state that the TM is in while looking for the leftmost 0, without having seen a 1, and q_L^0 is the state while it is looking for the leftmost 0 but after seeing at least one 1;

- q_L^0 is the state after the TM has found and crossed out the leftmost 0, and is moving back to the leftmost cell;
- q_1^1 is the state after the TM has found and crossed out the leftmost 0, has moved back to the leftmost cell, and is now looking for the first 1 to cross out;
- similarly, q_1^2 is the state while the TM is looking for the second 1 to cross out; and
- q_1^L is the state after the TM has found and crossed out the two 1s, and is moving back to the leftmost cell.

Answer to Question 2.

a. Yes, after $|x| + |Q|$ successive moves to the right, you can be certain that the TM will never halt.

This is because after $|x|$ successive moves to the right, the TM is in the region of cells that contain only blanks. After an additional at most $|Q|$ moves to the right, the TM must find itself in the same state, say q , as in some previous configuration in which it was scanning the blank cells. Therefore the cycle of states from q , through zero or more other states, back to q while scanning blank cells and moving to the right will keep repeating and the TM will keep moving to the right forever.

b. [ONLY IF] It suffices to show that if a language L is accepted by a DFSA A (i.e., if L is regular), there is a special TM M that recognizes L . The idea is for M to directly simulate A using A 's transition function and without ever changing any symbols on the tape, until its head is positioned over a blank cell, i.e., the TM has finished reading the input string. When that happens, if the TM is in a state that is in one of the DFSA's accept states, it enters its own accept state (and thus accepts the string); otherwise it enters its reject state. Thus, M accepts a string x if and only if A accepts x , and so the language accepted by A is recognized by the TM M .

Given the DFSA $A = (Q, \Sigma, \delta, q_0, F)$ it is straightforward to define formally the Turing machine $M = (Q', \Sigma, \Gamma, \delta', q_0, h_A, h_R)$ described above:

- $Q' = Q \cup \{h_A, h_R\}$, where h_A and h_R are states not in Q that are the accept and reject states of M ; note that the initial state q_0 of M is the same as that of A .
- $\Gamma = \Sigma \cup \{\sqcup\}$.
- $\delta'(q, a) = \begin{cases} (\delta(q, a), a, R), & \text{if } q \in Q \text{ and } a \neq \sqcup \\ (q_A, \sqcup, R), & \text{if } q \in F \text{ and } a = \sqcup \\ (q_R, \sqcup, R), & \text{otherwise (i.e., } q \in Q - F \text{ and } a = \sqcup) \end{cases}$

Note that M is special since it reads its input and never moves to the left.

[IF] It suffices to show that the language L recognized by a special TM M is also accepted by a FSA A (and therefore L is regular). The idea again is for A to simulate M .

To explain the simulation we define H_{empty} to be the set of states q of M such that, if we start M in state q on an empty tape (i.e., the input is ϵ), M eventually reaches the accept state. That is,

$$H_{\text{empty}} = \{q : \text{for some } x, y \in \Gamma^*, q \vdash_M^* xh_Ay\}.$$

Since M moves the tape head to the right in each of the first n moves on an input of length n , initially it behaves just like a FSA except that it may over-write the symbol on the cell before moving to the right. This, however, cannot affect the computation of M in the future, since M is a special TM and therefore it will never again look at the cells occupied by the input after reading it in its first n moves. After these n moves, M reaches the blank portion of the tape, which starts at cell $n + 1$. Let q be the state of M when it reaches cell $n + 1$ after its first n moves. M will never move to the left of cell $n + 1$, because it is a special TM. Therefore its future sequence of moves is exactly the same as if it started in state q on blank tape. So, by definition of H_{empty} , M will reach the accept state if and only if $q \in H_{\text{empty}}$. Therefore the FSA A

that simulates M enters an accept state if and only if after its first n moves it is in some state $q \in H_{\text{empty}}$. It is easy to see that the FSA A defined from M in this manner accepts a string if and only if M accepts that string, so the language recognized by M is accepted by A .

Given a special TM $M = (Q, \Sigma, \Gamma, \delta, q_0, h_A, h_R)$, we can define formally the FSA $A = (Q', \Sigma, \delta', q'_0, F)$ that simulates it as follows:

- $Q' = Q$ (i.e., the FSA has exactly the same states as the TM).
- $q'_0 = q_0$ (i.e., the FSA has the same initial state as the TM).
- $F = H_{\text{empty}} = \{q : \text{for some } x, y \in \Gamma^*, q \vdash_M^* xh_Ay\}$ (i.e., the FSA's accept states are the states from which M accepts on empty tape — note that this includes h_A).
- $\delta'(q, a) = \begin{cases} q', & \text{if } q \in Q - \{h_A, h_R\}, a \in \Sigma, \text{ and } \delta(q, a) = (q', b, R) \text{ for some } b \in \Gamma \\ q, & \text{otherwise (i.e., if } q = h_A \text{ or } q = h_R) \end{cases}$